RESEARCHMIND

A practical guide to the multi-agent AI research project

Search → Read → Write → Critique

PROJECT DOCUMENTATION

Architecture • Code walkthrough • Setup • Debugging • Interview preparation

What this document covers

This guide explains the current ResearchMind repository structure and the workflow visible in its code: a Streamlit interface, LangChain agents, Tavily search, webpage extraction, a report-writing chain, and a critic chain. It also includes setup guidance, practical testing steps, known limitations, and an interview-ready explanation.

Accuracy note: This document is based on the current source files visible in the public repository at the time of preparation. Provider behavior, free-tier limits, and model availability can change. The guide distinguishes implemented behavior from recommended future improvements.

1. Project at a glance

ResearchMind is a small multi-stage research assistant. A user enters a topic, and the application coordinates several AI-powered stages to gather information, inspect a source, produce a report, and evaluate that report.

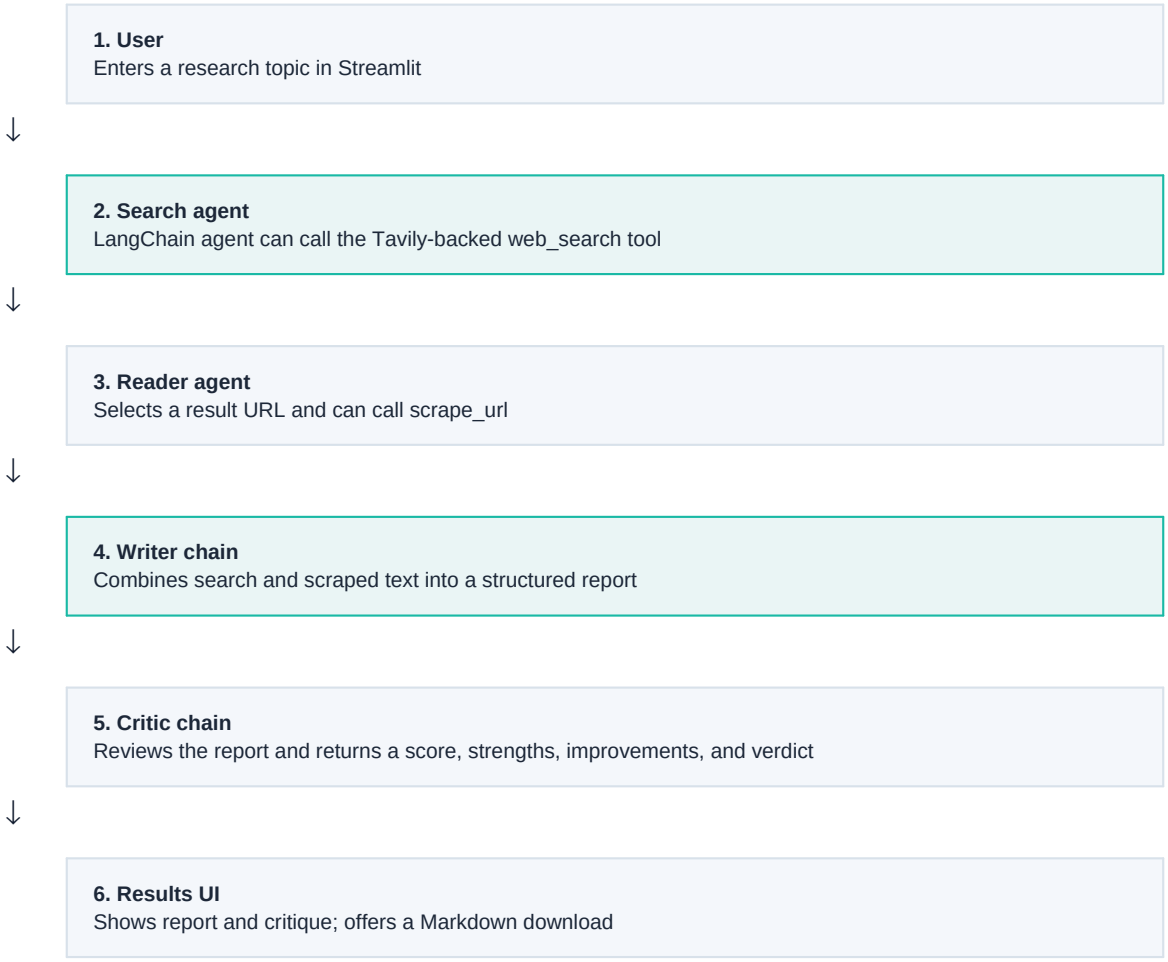
Area	Current implementation
User interface	Streamlit app with a dark visual theme, research topic input, pipeline status cards, expandable raw outputs, report display, and Markdown download.
Agent framework	LangChain's create_agent for the search and reader agents.
LLM provider	ChatOpenAI configured to call OpenRouter's OpenAI-compatible API endpoint using model name openrouter/free.
Search	TavilyClient, configured with TAVILY_API_KEY; requests up to five results.
Webpage extraction	requests + BeautifulSoup; removes script, style, nav, and footer elements; returns up to 3,000 characters.
Report generation	ChatPromptTemplate + LLM + StrOutputParser; asks for introduction, at least three key findings, conclusion, and sources.
Critique	A second prompt chain scores the report out of 10 and lists strengths, improvements, and a verdict.
Configuration	Dependencies declared in pyproject.toml and resolved through uv.lock; .env is loaded locally.

In one sentence: ResearchMind uses tool-enabled agents for discovery and reading, then deterministic prompt chains for report writing and critique.

This is a sequential pipeline, not a group of agents debating each other in parallel. Each stage depends on the output of the previous stage.

2. System architecture

The high-level flow below reflects the current app and the reusable command-line pipeline.



Important separation of responsibilities

- **app.py** handles the browser-facing interface and currently runs the stages directly when the user submits a topic.
- **pipline.py** (spelling as in the repository) exposes a reusable run_research_pipeline(topic) function and a terminal entry point.
- **agents.py** defines the LLM client, tool-using agents, and writer/critic chains.
- **tools.py** contains the external search and webpage-scraping tools.

3. The four stages in detail

Stage 1 — Search agent

The app calls build_search_agent() and invokes the resulting LangChain agent with a request to find recent, reliable, detailed information about the user's topic. The agent is given the web_search tool. That tool calls Tavily with max_results=5 and formats each result as a title, URL, and short snippet.

What comes out: a text response stored in the results dictionary under the key search. The app preserves this raw output in an expandable section so a user can inspect what the agent found.

Stage 2 — Reader agent

The app creates the reader agent with scrape_url as its available tool. It supplies the search output and asks the agent to select a relevant URL and scrape it. The tool makes an HTTP GET request with a timeout and browser-like User-Agent header, parses the HTML using BeautifulSoup, removes selected page elements, and extracts plain text.

Current constraint: the Streamlit app passes only the first 800 characters of the search output into the reader prompt. The standalone pipline.py passes up to 6,000 characters. This means the two entry points do not provide exactly the same context to the reader.

Stage 3 — Writer chain

The writer chain is a prompt template piped into the configured LLM and then StrOutputParser. It receives topic and research variables. The research string combines the search results and the reader's response. The prompt asks for an introduction, at least three well-explained key findings, a conclusion, and a list of source URLs. It also explicitly instructs the model not to invent sources or unsupported facts.

Stage 4 — Critic chain

The critic chain receives the generated report and asks for a strict review in a fixed format: Score: X/10, Strengths, Areas to Improve, and a one-line verdict. It is a separate LLM call. It does not automatically rewrite the report or change the final report; the critique is displayed as feedback.

4. Source-file walkthrough

agents.py — model, agents, and chains

- Loads environment variables with python-dotenv.
- Creates ChatOpenAI with model='openrouter/free', base_url='https://openrouter.ai/api/v1', API key from OPENROUTER_API_KEY, temperature=0, max_retries=1, and timeout=60.
- build_search_agent() calls create_agent with the web_search tool.
- build_reader_agent() calls create_agent with the scrape_url tool.
- writer_prompt and critic_prompt define the two text-generation tasks; StrOutputParser returns plain text.

Note: the current file imports ChatGroq as well, but the shown active llm variable is ChatOpenAI configured for OpenRouter. The unused import can be cleaned up later, but it is not required to understand the active configuration.

tools.py — external tools

```
tavily = TavilyClient(api_key=os.getenv("TAVILY_API_KEY"))

@tool
def web_search(query: str) -> str:
    results = tavily.search(query=query, max_results=5)
    # formats titles, URLs and snippets

@tool
def scrape_url(url: str) -> str:
    resp = requests.get(url, timeout=8, headers={"User-Agent": "Mozilla/5.0"})
    soup = BeautifulSoup(resp.text, "html.parser")
    # removes script, style, nav and footer
    return soup.get_text(separator=" ", strip=True)[:3000]
```

The @tool decorator exposes these Python functions to the LangChain agent with their names, argument schemas, and docstrings. The model can decide when to call an available tool, but the actual HTTP requests run in Python.

app.py — Streamlit UI

- Configures page title, icon, wide layout, CSS theme, hero section, topic input, and four pipeline cards.
- Uses st.session_state to retain results and flags across Streamlit reruns.
- When the user clicks Run, it clears old results, marks the app as running, and reruns. The pipeline then runs each stage inside a spinner.
- Results are saved stage-by-stage, so partial outputs can remain visible if a later stage fails.
- Shows search and reader text in expanders, report and critic feedback in styled sections, and a Markdown download button.

pipline.py — reusable pipeline entry point

Defines run_research_pipeline(topic) and validates that the topic is not empty. It executes the same four conceptual stages and returns a dictionary with topic, search_results, scraped_content, report, and feedback. Running the file directly prompts for a topic in the terminal.

pyproject.toml and uv.lock

pyproject.toml declares the project dependencies and Python requirement. uv.lock records a resolved dependency set so environments can be reproduced more consistently. When these files are used by deployment, update and commit them together after dependency changes.

5. What happens when a user clicks Run?

Step	Action	State / result
1	Validate the topic	If blank, show a warning and stop.
2	Reset previous run	results becomes an empty dictionary; running=True; done=False.
3	Search	Call search agent; save results['search'].
4	Read	Call reader agent with a portion of search output; save results['reader'].
5	Write	Combine the evidence text; call writer_chain; save results['writer'].
6	Critique	Call critic_chain on the report; save results['critic'].
7	Finish	Set running=False and done=True; rerun to render final state.
8	Failure path	Catch an exception, stop running, show an error and troubleshooting hint.

Why session state matters

Streamlit reruns the Python script after many interactions. st.session_state lets the app keep the current topic, result dictionary, and pipeline flags between those reruns. Without it, a rerun could reset the UI to an empty state.

One implementation detail to understand

The pipeline cards are rendered before the long-running stages execute. The spinner messages show progress during the run, and results are saved stage by stage, but the cards are not updated in real time after every stage unless the app rerenders those card elements. Treat the cards as a status overview rather than a precise live progress bar.

6. Local setup and configuration

Prerequisites

- Python version compatible with pyproject.toml (requires Python 3.12 or newer).
- Git and uv installed, if you intend to manage the project using the current lockfile workflow.
- An OpenRouter API key and a Tavily API key.

Suggested Windows PowerShell setup

```
# Open the project directory
cd "C:\Users\admin\Downloads\multi agent"

# Create and activate a virtual environment (if needed)
uv venv .venv
.\.venv\Scripts\Activate.ps1

# Install the project's declared dependencies
uv sync

# Run the Streamlit app
python -m streamlit run app.py
```

Environment variables

Create a local .env file in the project root. Never commit this file or paste its secret values into public logs.

```
OPENROUTER_API_KEY=your_openrouter_key_here
TAVILY_API_KEY=your_tavily_key_here
```

The source code uses load_dotenv() to load local variables. For Streamlit Community Cloud, add the variables in the app's Secrets settings rather than relying on a local .env file.

If dependency installation fails

- Check that you are in the folder containing pyproject.toml and uv.lock.
- Run uv sync again after editing dependencies.
- If you install a package manually, also add it to the project dependency declaration and regenerate the lockfile so deployment uses the same package set.
- Do not commit .venv, cache directories, API keys, or large local model files.

7. Deployment, security, and reliability

Secrets

OPENROUTER_API_KEY is read by agents.py; TAVILY_API_KEY is read by tools.py. Both are required for the current online path. If either key is missing or invalid, the associated provider/tool call will fail. Keep credentials in .env locally and deployment secrets in Streamlit Cloud.

Provider quotas and availability

The configured OpenRouter model identifier is openrouter/free. Free routing can vary by availability, rate limits, context length, and tool-calling support. A failure may come from the provider/model route rather than the Streamlit UI. Inspect the full exception in local terminal or Streamlit logs, then verify the selected model's current capabilities and quota.

Web scraping limitations

- Some sites block automated requests, require JavaScript, or return incomplete content.
- The current scraper uses a simple requests call and does not execute JavaScript.
- It truncates extracted content to 3,000 characters, so important evidence farther down a page may be omitted.
- The code does not currently check response status codes with raise_for_status() or validate the URL against private/local network targets.

Research quality limitations

- The search stage is limited to up to five Tavily results.
- The reader stage is instructed to choose a relevant URL; it does not systematically read every result.
- The report prompt asks for sources, but source accuracy still needs human checking against the original URLs.
- The critic provides a textual review, not a guaranteed factual verification. It may miss errors or criticize valid points.
- The app currently does not have persistent report storage, user accounts, citations with line-level evidence, or a human approval workflow.

Practical improvements to consider later (not required to run the current project)

- Add HTTP status validation and a clearer error message for blocked or empty webpages.
- Validate and constrain URLs before making outbound requests.
- Preserve a structured list of source URLs alongside every extracted claim.
- Add request timeouts and stage-specific logging without exposing secrets.
- Use an explicit model that supports the required tool-calling format if free routing is unreliable.
- Add small tests for blank topics, tool output formatting, scraper failures, and report schema.

8. Troubleshooting checklist

Symptom	Likely checks
OPENROUTER_API_KEY missing / authentication error	Confirm the variable name exactly matches agents.py; verify the key is active; check Streamlit Cloud Secrets for deployed app.
TAVILY_API_KEY error	Confirm Tavily key exists, is valid, and is available in the same environment running the app.
429 / rate limit	Provider quota or rate limit. Wait or use a currently supported model/provider route; avoid repeatedly retrying expensive calls.
Model fails to call tools	Check whether the selected route supports tool calling in the LangChain/OpenAI-compatible format; try a known supported model route.
Search works, reader fails	The URL may block scraping, time out, or return content that cannot be parsed. Inspect the reader output and logs.
Report is empty or weak	Inspect search and reader expanders; verify those stages returned usable evidence before the writer stage.
ModuleNotFoundError	Activate the intended .venv, run uv sync from the project root, and check that the dependency is declared in pyproject.toml.
Cloud uses old code	Confirm the correct branch was pushed, inspect the latest GitHub commit, and check Streamlit Cloud's app repository/branch settings.
Git says nothing to commit	Run git status; stage changed files with git add; commit; then push to the configured branch.

Useful commands

```
git status
git add agents.py app.py tools.py pipline.py pyproject.toml uv.lock test.py
git commit -m "Update ResearchMind"
git push origin main

# Syntax check key Python files
python -m py_compile app.py agents.py tools.py pipline.py
```

Only stage files that you actually intend to publish. Before committing, inspect git status and confirm .env and any secret files are not staged.

9. A practical test plan

Test	How to test	Expected result
Blank topic	Click Run with the input empty.	Warning appears; no API call should be needed.
Basic topic	Use a narrow topic with well-known public information.	Search, reader, writer, and critic stages finish.
Partial failure	Temporarily use an invalid key in a safe local test environment.	App shows an error and does not pretend the run succeeded.
Scraper failure	Test a URL that returns an error or blocks requests.	Tool returns a failure message; check how the reader handles it.
Download	Complete a run and click Download report.	A Markdown file downloads containing the writer output.
Repeat run	Run a second topic in the same session.	Previous results are cleared and new results replace them.
Deployment smoke test	Push changes and run the deployed app with secrets configured.	UI loads, provider calls work, and report download works.

Manual evaluation checklist for report quality

- Does each key finding connect to evidence in the supplied source material?
- Are the source URLs real and relevant to the claim they support?
- Are facts, dates, numbers, and quotations verified against the original pages?
- Does the report distinguish confirmed facts from uncertainty or interpretation?
- Does the critic identify concrete gaps rather than only giving a score?

10. Interview-ready explanation

30-second project pitch

> *“ResearchMind is a Streamlit-based multi-agent research assistant built with LangChain. A user enters a topic, a search agent retrieves relevant web results through Tavily, a reader agent extracts deeper text from a selected source, a writer chain creates a structured report, and a critic chain evaluates that report. The app displays intermediate outputs and lets the user download the final report as Markdown. I designed it as a sequential pipeline so each stage has a clear responsibility.”*

Likely interview questions

Q. Why use agents instead of one prompt?
A. The search and reader stages need tools and can decide when to call them. The writer and critic stages are more constrained tasks, so simple prompt chains are sufficient for the current design.

Q. What is the role of Tavily?
A. Tavily provides search results such as titles, URLs, and snippets. The tool wrapper formats those results for the agent; Tavily is not the language model writing the report.

Q. What does the @tool decorator do?
A. It makes a Python function available to a LangChain agent as a callable tool with a name, description, and argument schema.

Q. What is the difference between an agent and a chain here?
A. The search/reader agents can select and call available tools. The writer/critic chains follow a defined prompt → model → output parser sequence.

Q. Why use StrOutputParser?
A. It converts the model response into plain text for downstream display and storage.

Q. Why is st.session_state used?
A. Streamlit reruns the script on interaction. Session state preserves the pipeline results and run flags across reruns.

Q. Does the critic guarantee factual correctness?
A. No. It is a model-generated review. Claims and sources still need verification, and the critic can miss mistakes.

Q. What would you improve next?
A. Stronger URL validation, source-to-claim traceability, status-code handling, better tests, and more reliable model routing with explicit tool-calling support.

11. Quick glossary

Term	Plain-English meaning
LLM	A language model that generates or transforms text.
Agent	A model-driven component that can choose to call from a set of tools to complete a task.
Tool	A Python function exposed to the agent, such as web search or URL scraping.
Prompt template	A reusable instruction with placeholders filled at runtime.
Chain	A sequence of processing steps; here, prompt → model → output parser.
StrOutputParser	Converts a model output into plain text.
Session state	Streamlit's per-session storage for values that must survive reruns.
Environment variable	A configuration value loaded at runtime, commonly used for API keys.
Lockfile	A record of resolved dependency versions for reproducible installation.
Scraping	Fetching a webpage and extracting readable text from its HTML.
Critique	A generated review of the report; useful feedback, but not a formal verification system.

Final takeaway

ResearchMind demonstrates several portfolio-relevant concepts in one understandable workflow: tool-calling agents, external search, webpage text extraction, prompt chaining, output parsing, Streamlit state management, environment-based secrets, and dependency locking. In an interview, be clear about what is implemented today, what is only a possible improvement, and why the system still requires human verification of sources and claims.

Repository: https://github.com/siddhant754962/ResearchMind